Program 6: N-gram Model for Text Generation

```
import random

text = "i love natural language processing i love pythor

words = text.split()

# Create bigrams
bigrams = {}

for i in range(len(words) - 1):
    first = words[i]
    second = words[i + 1]

    if first not in bigrams:
        bigrams[first] = []

    bigrams[first].append(second)

# Generate text
current = "i"
generated = [current]

for i in range(6):
    if current in bigrams:
        current = random.choice(bigrams[current])
        generated.append(current)
    else:
        break

print("Generated Text:", " ".join(generated))
```

```
Generated Text: i love natural language processing i love
```

Program 7: Part-of-Speech Tagging Using NLTK

```
import nltk

# Download required resources
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')

text = "The boy plays cricket in the park"

# Tokenize the sentence
words = nltk.word_tokenize(text)

# Perform POS tagging
tags = nltk.pos_tag(words)

print("Word and POS Tags:")

for word, tag in tags:
    print(word, "->", tag)
```

```
Word and POS Tags:
The -> DT
boy -> NN
plays -> VBZ
cricket -> NN
in -> IN
the -> DT
park -> NN
```

Program 8: Stochastic Part-of-Speech Tagging

```python
# Training data
training_data = [
    [("the", "DT"), ("dog", "NN"), ("runs", "VBZ")],
    [("the", "DT"), ("cat", "NN"), ("sleeps", "VBZ")],
    [("a", "DT"), ("boy", "NN"), ("plays", "VBZ")]
]

# Calculate word and tag frequencies
word_tags = {}
tag_counts = {}

for sentence in training_data:
    for word, tag in sentence:
        word_tags.setdefault(word, {})
        word_tags[word][tag] = word_tags[word].get(tag, 0) + 1
        tag_counts[tag] = tag_counts.get(tag, 0) + 1

# Calculate probabilities and assign tags
def stochastic_tagging(words):
    result = []

    for word in words:
        if word in word_tags:
            tag = max(word_tags[word],
                      key=word_tags[word].get)
        else:
            tag = "NN"
            result.append((word, tag))

    return result


text = input("Enter a sentence: ")
words = text.lower().split()

tags = stochastic_tagging(words)

print("Stochastic POS Tagging:")
for word, tag in tags:
    print(word, "->", tag)
```

```
Enter a sentence: kohli goes down the ground kohli goes out of the ground
Stochastic POS Tagging:
kohli -> NN
goes -> NN
down -> NN
the -> DT
ground -> NN
kohli -> NN
goes -> NN
out -> NN
of -> NN
the -> DT
ground -> NN
```

Program 9: Rule-Based POS Tagging Using Regular Expressions

```python
import nltk

# Define regular expression rules
patterns = [
    (r'.*ing$', 'VBG'),
    (r'.*ed$', 'VBD'),
    (r'.*ly$', 'RB'),
    (r'.*ous$', 'JJ'),
    (r'.*', 'NN')
]

# Create a regular expression tagger
tagger = nltk.RegexpTagger(patterns)

text = "playing worked quickly famous student"

words = text.split()

# Perform POS tagging
tags = tagger.tag(words)

print("Rule-Based POS Tagging:")

for word, tag in tags:
    print(word, "->", tag)
```

```
Rule-Based POS Tagging:
playing -> VBG
worked -> VBD
quickly -> RB
famous -> JJ
student -> NN
```

Program 10: Transformation-Based Tagging

```python
# Initial POS tags
words = ["He", "is", "playing", "cricket", "today"]

tags = ["NN", "VBZ", "NN", "NN", "NN"]

print("Before Transformation:")

for word, tag in zip(words, tags):
    print(word, "->", tag)

# Transformation rules

for i in range(len(words)):

    # Rule 1: Words ending with 'ing' are tagged VBG
    if words[i].endswith("ing"):
        tags[i] = "VBG"

    # Rule 2: Words following 'is' are tagged as VBG
    if i > 0 and words[i-1].lower() == "is":
        tags[i] = "VBG"

    # Rule 3: 'today' is tagged as a noun
    if words[i].lower() == "today":
        tags[i] = "NN"

print("\nAfter Transformation:")
```

```python
    for word, tag in zip(words, tags):
        print(word, "->", tag)
```

```
Rule-Based POS Tagging:
playing -> VBG
worked -> VBD
quickly -> RB
famous -> JJ
student -> NN
```